

SOEN 6011 — Delivery 1

Function F7: x^y

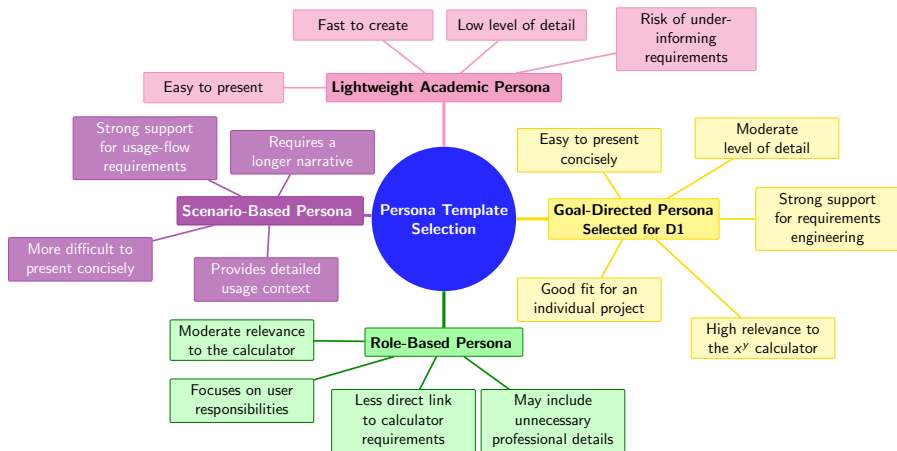
Yichen Huang
Student ID: 40167688

Concordia University

July 15, 2026

- ➊ Problem 1: Persona Selection and Profile
- ➋ Problem 2: Requirements
- ➌ Problem 3: Candidate Algorithms
- ➍ Problem 4: Algorithm Selection and Implementation

Problem 1 — Persona Template Selection



Decision and Persona Basis

The goal-directed persona was selected because the user's goals can be translated directly into clear, traceable, and verifiable requirements.

Alex Chen is a fictional, assumption-based persona created from the expected usage context of undergraduate students solving mathematical and engineering problems involving power expressions.

Problem 1 — User Persona Profile

Alex Chen



Photo: RDNE Stock project

/ Pexels, no. 7683899.

*Fictional, assumption-based
persona*

Second-Year Engineering Student Technical Level

Intermediate computer user
Beginner programmer

Primary Context

Homework and laboratory
work

Background

Alex solves mathematics, physics, and engineering problems involving power expressions with integer and rational exponents.

Main Goals

- Calculate supported power expressions accurately.
- Understand why an input is rejected.
- Continue using the program after invalid input.

Main Needs

- Separate prompts for the base x and exponent y .
- Clearly labelled results.
- Finite-input and supported-domain validation.
- Plain-language error messages.

Problem 2 — Assumptions

Scope Assumptions

- The calculator produces **real-valued output only**.
- The base x and exponent y must be finite numeric values.
- For $x > 0$, finite integer and rational-style decimal exponents are supported.
- For $x = 0$, the expression is supported only when $y > 0$.
- For $x < 0$, the exponent must be integer-valued.
- Negative bases with non-integer exponents are outside the D1 scope, even when some rational cases may have real-valued results.
- Non-numeric input, NaN, infinity, overflow, underflow to an unrepresentable result, and other non-finite results are treated as invalid or unsupported.
- Rational-style exponents are entered in decimal form.
- The implementation uses a Python textual user interface.

Problem 2 — Supported Domain

Positive Base: $x > 0$

Finite integer and non-integer decimal exponents are supported.

Examples:

$$2^3 = 8, \quad 4^{0.5} = 2, \quad 16^{0.25} = 2$$

Zero Base: $x = 0$

The expression is supported only when $y > 0$.

Therefore, 0^3 is supported, while 0^0 and 0^{-1} are rejected.

Negative Base: $x < 0$

Negative bases are supported only when y is integer-valued.

Some negative bases with non-integer rational exponents may have real-valued results. However, these cases are outside the D1 scope and are rejected with a clear error message.

Scope Decision

This simplification provides consistent domain handling across the requirements, algorithms, implementation, and tests.

Mathematical domain based on standard properties of exponential expressions; negative-base simplification confirmed as acceptable for D1.

Problem 3 — Algorithm 1: Hybrid Power Evaluation

Purpose

Compute x^y within the supported real-valued domain by selecting an appropriate computational method based on the exponent.

Input and Output

Input: Finite real values x and y .

Output: A real-valued result or a clear error message.

Preconditions

- x and y are finite numeric values.
- If $x = 0$, then $y > 0$.
- If $x < 0$, then y is integer-valued.

Method Selection

- Integer-valued y : exponentiation by squaring.
- Non-integer y with $x > 0$: logarithm-exponential transformation.

Language-Independent Pseudocode

```
ALGORITHM HybridPowerEvaluation( $x, y$ )
INPUT: finite real numbers  $x, y$ 
OUTPUT: result or error
IF  $x = 0$  THEN
    IF  $y > 0$  THEN
        RETURN 0
    ELSE
        RETURN ERROR
    END IF
END IF
IF  $y$  is integer-valued THEN
    result  $\leftarrow$  PowerBySquaring( $x$ , integer( $y$ ))
ELSE
    IF  $x < 0$  THEN
        RETURN ERROR
    END IF
    result  $\leftarrow \exp(y \times \ln(x))$ 
END IF
IF result is non-finite THEN
    RETURN ERROR
END IF
RETURN result
END ALGORITHM
```

Requirements Supported

FR-001, IR-003, CR-001, ER-001, ER-002, ER-003, OR-001, and AR-001.

Integer-exponent path uses exponentiation by squaring. For $x > 0$, the non-integer path uses $x^y = e^{y \ln(x)}$.

Problem 3 — Algorithm 2: Library-Assisted Power Evaluation

Purpose

Compute x^y after validating the supported real-valued domain, using a language-provided power operation.

Input and Output

Input: Finite real values x and y .

Output: A real-valued result or a clear error message.

Preconditions

- x and y are finite numeric values.
- If $x = 0$, then $y > 0$.
- If $x < 0$, then y is integer-valued.

Limitation

The computational details are hidden inside the language-provided power operation.

Language-Independent Pseudocode

```
ALGORITHM LibraryPowerEvaluation( $x, y$ )
INPUT: finite real numbers  $x, y$ 
OUTPUT: result or error
IF  $x = 0$  AND  $y \leq 0$  THEN
    RETURN ERROR
END IF
IF  $x < 0$  AND  $y$  is not integer-valued THEN
    RETURN ERROR
END IF
TRY
    result  $\leftarrow$  language-provided power operation( $x, y$ )
CATCH domain or overflow error
    RETURN ERROR
END TRY
IF result is non-finite THEN
    RETURN ERROR
END IF
RETURN result
END ALGORITHM
```

Requirements Supported

FR-001, CR-001, ER-001, ER-002, ER-003, OR-001, and AR-001.

Candidate based on a language-provided power operation after explicit domain and numeric-range validation.

Problem 4 — Algorithm Selection

Criterion	Algorithm 1: Hybrid Power Evaluation	Algorithm 2: Library-Assisted Evaluation
Supported Domain	Supports the complete defined D1 domain: integer-valued exponents for all valid bases and non-integer decimal exponents for positive bases.	Supports the same domain after explicit input and domain validation.
Integer-Exponent Method	Uses exponentiation by squaring.	Uses a language-provided power operation.
Non-Integer Method	Uses the logarithm-exponential transformation when $x > 0$.	Uses a language-provided power operation.
Computational Cost	The integer path requires $O(\log y)$ multiplications.	The internal computational cost is hidden by the language implementation.
Algorithm Transparency	The computational steps are explicit and explainable.	The actual calculation is hidden inside the language-provided operation.
Numerical and Range Handling	Intermediate and final results can be checked explicitly for overflow and non-finite values.	Domain and overflow errors are handled after calling the built-in operation.
D1 Suitability	Selected	Not selected

Problem 4 — Computational Flow

Hybrid Evaluation Flow

Read and validate x and y



Is $x = 0$?

If $y > 0$, return 0; otherwise, display a domain error.



Is y integer-valued?

Yes: use exponentiation by squaring.

No: if $x > 0$, use $\exp(y \ln x)$; otherwise, display a domain error.



Check overflow and non-finite result



Display result or error message

The integer and non-integer paths implement the same supported-domain rules defined in Problem 2.

Problem 4 — Manual Verification

Base x	Exponent y	Observed Result	
2	3	Result: 8	Passed
2	-3	Result: 0.125	Passed
4	0.5	Result: 2	Passed
-2	3	Result: -8	Passed
0	0	Rejected because 0^0 is undefined	Passed
0	-3	Rejected: zero cannot be raised to a negative exponent	Passed
-2	0.5	Rejected: a negative base requires an integer exponent	Passed
nan/inf	-	Rejected non-finite input and re-prompted	Passed
10^{308}	2	Reported numeric-range overflow	Passed
10^{-300}	2	Reported unrepresentable underflow	Passed

Verification Result

All representative test cases produced the expected result or a clear error message. The program remained interactive after invalid input and exited normally when the user selected `n`.

Problem 4 — Terminal Test Evidence

```
PS D:\Concordia_master\comp_6011\DI> dir
PS D:\Concordia_master\comp_6011\DI> py -a src.main
Base function Calculator: &?y
Enter finite numeric values, type q, quit, or exit to leave.
Enter the base x: 2
Enter the exponent y: 3
Result: 8

Would you like to perform another calculation? (y/n): y

Enter the base x: 2
Enter the exponent y: -3
Result: 0.125

Would you like to perform another calculation? (y/n): y

Enter the base x: 4
Enter the exponent y: 0.5
Result: 2

Would you like to perform another calculation? (y/n): y

Enter the base x: -2
Enter the exponent y: 3
Result: -8

Would you like to perform another calculation? (y/n): y

Enter the base x: 0
Enter the exponent y: 0
Error: exp is undefined in this calculator.

Would you like to perform another calculation? (y/n): y
```

Valid integer, negative-exponent,
decimal-exponent, negative-base, and
zero-base cases.

```
Enter the base x: 0
Enter the exponent y: 3
Result: 0

Would you like to perform another calculation? (y/n): y

Enter the base x: 0
Enter the exponent y: -3
Error: zero cannot be raised to a negative exponent.

Would you like to perform another calculation? (y/n): y

Enter the base x: -2
Enter the exponent y: 0.5
Error: a negative base requires an integer-valued exponent.

Would you like to perform another calculation? (y/n): y

Enter the base x: abc
Error: the base and exponent must be numeric values.
Enter the base x: 2
Enter the exponent y: abc
Error: the base and exponent must be numeric values.
Enter the exponent y: 3
Result: 8

Would you like to perform another calculation? (y/n): y
```

Undefined cases, invalid domains, and
non-numeric input handling.

```
Enter the base x: nan
Error: NaN and infinity are not supported.
Enter the base x: inf
Error: NaN and infinity are not supported.
Enter the base x: -inf
Error: NaN and infinity are not supported.
Enter the base x: 2
Enter the exponent y: inf
Error: NaN and infinity are not supported.
Enter the exponent y: 3
Result: 8

Would you like to perform another calculation? (y/n): y

Enter the base x: 1e308
Enter the exponent y: 2
Error: the result is outside the supported numeric range.

Would you like to perform another calculation? (y/n): y

Enter the base x: 1e-300
Enter the exponent y: 2
Result: 0

Would you like to perform another calculation? (y/n): 2
Please enter y to continue or n to exit.
Would you like to perform another calculation? (y/n): n
Goodbye.
PS D:\Concordia_master\comp_6011\DI>
```

NaN and infinity rejection, overflow detection,
underflow behaviour, continuation validation,
and graceful exit.

GAI Use Across Delivery 1

Purpose of GAI Use

GAI tools supported brainstorming, comparison, organization, and first-draft generation. All outputs were reviewed and revised before use.

Part	Prompt Type	Purpose	Manual Review
P1	Ideation and decision support	Compare persona templates and generate candidate user profiles.	Checked realism, relevance, and connection to requirements.
P2	Requirements elicitation and specification	Draft assumptions and uniquely identified requirements.	Checked clarity, feasibility, traceability, and verifiability.
P3	Algorithm generation and comparison	Generate two language-independent algorithms for x^y .	Checked domain correctness, pseudocode independence, and requirement coverage.
P4	Decision-making and code generation	Compare algorithms and draft the Python textual interface.	Tested code, corrected domain handling, and simplified the implementation.

Student Responsibility

GAI outputs were treated as drafts, not authoritative sources. The final persona, requirements, algorithms, selection, and implementation were verified and revised by the student.

GAI Output Comparison and Final Decisions

GAI Tool	Useful Contribution	Issue Identified	Final Decision
ChatGPT	Clear slide structure and concise wording.	Some requirements and mathematical assumptions were oversimplified.	Used for presentation structure and concise wording.
Claude	Strong persona-to-requirement traceability and input-validation logic.	Algorithm 2 partly depended on Algorithm 1; some domain wording needed correction.	Used mainly for persona, requirements, and code organization.
Gemini	Detailed terminal interaction and professional wording.	Persona choice was inconsistent and the selected algorithm had a narrower domain.	Only selected interface ideas and wording were retained.

Final Integrated Decisions

- Goal-directed engineering-student persona
- ISO/IEC/IEEE 29148-style "The system shall..." requirements
- Direct Power and Logarithm-Exponential algorithms
- Direct Power selected for D1
- Python textual interface with domain validation

References



Concordia University.

SOEN 6011 Project Description.

Course material provided through Moodle, 2026.



ISO/IEC/IEEE.

ISO/IEC/IEEE 29148:2018: Systems and Software Engineering—Life Cycle Processes— Requirements Engineering.

ISO Standard 72089.



Tavakoli, H.

"The CASTROFF Framework: A Professional Standard for Prompt Engineering."

In *Prompt Engineering for Everyone*, Apress, 2026.

DOI: 10.1007/979-8-8688-2338-1_6.



Python Software Foundation.

"The Power Operator."

Python Language Reference: Expressions.

Python Expressions Documentation.



Python Software Foundation.

math—Mathematical Functions.

Python Standard Library Documentation.



Interaction Design Foundation.

"Personas—A Simple Introduction."

Interaction Design Foundation.



OpenStax.

Precalculus 2e.

Sections on exponential and logarithmic functions.

OpenStax Precalculus 2e.



Milanović, M.

Laws of Software Engineering.

lawssoftwareengineering.com.

Thank You

Backup Slides

Detailed requirements, algorithms, tests, and references

Problem 1 — Persona Goals and Needs

Goals

- Compute x^y quickly and accurately.
- Obtain real-valued results for supported inputs.
- Understand why an input is rejected.

Needs

- Clear prompts for x and y .
- Clearly labelled output.
- Numeric input validation.
- Plain-language error messages.

Pain Points

- Unclear “Math Error” messages.
- Unclear handling of unsupported domain cases.
- Programs that stop after invalid input.

Influence on Problem 2

The persona leads to requirements for guided input, real-valued output, numeric validation, clear domain handling, understandable error messages, and a textual user interface.

Problem 2 — Requirement Style and Quality

Selected Requirement Style

The requirements use the ISO/IEC/IEEE 29148-style “The system shall...” structure.

Requirement Quality Criteria

Each requirement is:

- clear and unambiguous;
- feasible within the D1 scope;
- verifiable through testing or inspection;
- traceable to the persona or project assumptions;
- assigned a unique identifier.

Requirement Categories

FR: Functional	IR: Input	OR: Output
ER: Error Handling	UR: Usability	
AR: Accuracy	CR: Constraint	

Problem 2 — Traceability and Identifier Logic

Traceability Logic

Persona Need or Assumption → **System Requirement** → **Verification Method**

Identifier Uniqueness

Each requirement identifier uses:

- a category prefix;
- a sequential number;
- a value assigned once and never reused.

Examples

FR-001

IR-001

ER-001

AR-001

Example Traceability

Alex needs clear error messages →

ER-001: The system shall display an error message for an unsupported input →
Verification: Test a predefined unsupported input

Problem 2 — Output and Error Requirements

ID	Requirement Statement	Persona Link	Verification
OR-001	The system shall display the computed result with a clear label.	Alex needs unambiguous output.	Inspect the displayed result.
ER-001	The system shall display an error message when $x = 0$ and $y \leq 0$.	Alex needs clear boundary handling.	Test 0^0 and 0^{-1} .
ER-002	The system shall display an error message when $x < 0$ and y is not integer-valued.	Alex needs predictable real-valued behavior.	Test a negative base with a non-integer exponent.
ER-003	The system shall display an error message when the computed result overflows or becomes non-finite.	Alex needs reliable failure handling.	Test values that exceed the supported numeric range.
ER-004	The system shall recover from invalid input without terminating unexpectedly.	Alex wants the program to continue after an error.	Enter invalid input followed by valid input.

Requirement wording adapted from ISO/IEC/IEEE 29148:2018.

Problem 2 — Usability and Accuracy Requirements

ID	Requirement Statement	Persona Link	Verification
UR-001	The system shall use plain-language prompts and error messages.	Alex is a beginner programmer.	Inspect the wording.
UR-002	The system shall use a textual user interface for input and output.	Alex can use a terminal-based tool.	Run the program in a terminal.
UR-003	The system shall allow repeated calculations until the user chooses to exit.	Alex needs continued use after each calculation.	Perform multiple calculations and exit gracefully.
AR-001	For predefined independently verified test cases, the computed result shall have an absolute error of at most 10^{-9} when the expected result is zero, and a relative error of at most 10^{-9} otherwise.	Alex needs trustworthy results.	Compare with manually verified values or an independent trusted reference.

Independent Validation Rule

The implementation shall not validate its result using the same Python power operation used for the calculation.

Requirement wording and verification principles adapted from ISO/IEC/IEEE 29148:2018.

Problem 4 — Python Textual UI Implementation

Selected Algorithm

Hybrid Power Evaluation

Program Structure

- Read the base x .
- Read the exponent y .
- Validate finite numeric input.
- Determine whether y is integer-valued.
- Check the supported real-valued domain.
- Use exponentiation by squaring for integer-valued y .
- Use $\exp(y \ln x)$ for $x > 0$ and non-integer y .
- Check overflow and non-finite results.
- Display a result or a clear error message.
- Repeat until the user chooses to exit.

Textual User Interface

The program uses terminal prompts, plain-language messages, input recovery, repeated calculations, and an explicit exit option.

Requirements-to-Code Traceability

ID	Implementation Element
FR-001	Hybrid evaluation computes x^y within the supported domain.
IR-002	<code>math.isfinite()</code> validates x and y .
IR-003	<code>y.is_integer()</code> identifies integer-valued exponents.
IR-004	Input-validation loops re-prompt after invalid input.
ER-001	The program rejects 0^0 and zero with a negative exponent.
ER-002	The program rejects a negative base with a non-integer exponent.
ER-003	Exceptions and finite-result checks handle overflow and invalid results.
UR-001	Prompts and error messages use plain language.
UR-003	The main loop supports repeated calculations and graceful exit.
AR-001	Results are checked against independently verified expected values.